ECE C147/C247, Winter 2025 **Midterm Solution**
Neural Networks & Deep Learning Prof. J.C. Kao
UCLA ECE TAs: T. Monsoor, B. Qu, K. Pang, S. Dong, S. Rajesh, X. Yan

UCLA True Bruin academic integrity principles apply.
Open: Four pages of cheat sheet allowed, Calculator.
Closed: Book, computer, Internet.
2:00pm-3:50pm PT.
Wednesday, 19 Feb 2025.

State your assumptions and reasoning.
No credit without reasoning.
Show all work on these pages.

Name: _____

Signature: _____

ID#: _____

Problem 1	_____ / 24
Problem 2	_____ / 31
Problem 3	_____ / 20
Problem 4	_____ / 25
BONUS	_____ / 7 bonus points
Total	_____ / 107 points

1. **Multiple choice and short answer.** (20 points)

(a) (4 points) You are training a softmax classifier, as we learned in class, to classify 28×28 black-and-white images into one of 7 possible classes. Assume that the weights and biases are randomly and properly initialized and the loss is the cross-entropy loss. Approximately what value should you expect the loss to be after the first forward pass?

(A) 2.3
(B) 0.81
(C) 1.9
(D) 7.0
(E) There is not enough information to answer this question.

Solution: (C), $\ln(7) \approx 1.945$.

(b) (4 points) You find that your softmax classifier is underfitting the training data and performing poorly on the validation set. Which of the following could help mitigate underfitting while also improving generalization to unseen data? Select all that apply.

(A) Add dropout during validation.
(B) Add the Frobenius norm of the weight matrix to the loss function.
(C) Remove all biases from your model.
(D) Redo the train-val-test split and assign less data as training data.
(E) None of the above.

Solution: (E).

(c) (12 points) You are developing an image classification model and considering using the Fully Connected Neural Network (FCNN). Initially, you use the SGD optimizer with a fixed learning rate to train two FCNNs with different model sizes and obtain the following training and validation accuracies:

	Training Accuracy	Validation Accuracy
Model A	0.96	0.77
Model B	0.69	0.62

(Both of the following questions (i) and (ii) are based on this context.)

i. (6 points) Your teammate suggest adding Batch Normalization (BN) to Model A. In no more than three sentences, discuss whether BN can improve Model A's performance and explain why.

Solution: Model A has a high training accuracy and a low validation accuracy, which indicates potential overfitting. Batch Normalization (BN) may improve Model A's performance by stabilizing training and reducing internal covariate shift. If the model is indeed overfitting, BN can also be seen as a form of regularization by introducing constraints and noise during training to improve generalization.

ii. (6 points) You observe that the training loss of Model B oscillates significantly without decreasing consistently. In no more than 3 sentences, provide a suggestion to mitigate this issue and explain why it would be effective.

Solution: We can decrease the learning rate, using a smaller learning step to stabilize the weight updates. Another approach is using an adaptive optimizer, such as Adam, RMSprop, etc, which adjusts learning rates dynamically for each parameter, leading to smoother training loss decreasing.

(d) (4 points) Finally, you have decided to use a Convolutional Neural Network (CNN) that you learned in class to perform the image classification task described in (a). You implemented a CNN with the following architecture:

$$\texttt{conv} - \texttt{relu} - 2 \times 2\,\texttt{maxpool} - \texttt{affine}_1 - \texttt{relu} - \texttt{affine}_2 - \texttt{softmax}$$

`conv` is a convolution layer with five `5x5` filters, `stride = 1`, and `padding = 1`.

$\texttt{affine}_1$ is a fully-connect layer of dimension $\texttt{h}_1 \times 128$.

$\texttt{affine}_2$ is a fully-connect layer of dimension 128×7.

2×2 `maxpool` layer has `stride = 2`

Note: the output of the 2×2 `maxpool` is a matrix, which is reshaped into a vector of length $\texttt{h}_1$ prior to $\texttt{affine}_1$.

Compute the value of $\texttt{h}_1$ (i.e., the number of output neurons after the $2{\times}2$ `maxpool`) and the number of parameters for the layers shown below and fill in the blanks. You may receive partial credit for correct work.

$\texttt{h}_1 =$ [ ], Number of parameters in `conv` = [ ]

Number of parameters in $2 \times 2\,\texttt{maxpool}$ = [ ]

Number of parameters in $\texttt{affine}_1$ = [ ]

Solution: $\texttt{h}_1 = 845$. `conv` layer has $5 * (5 * 5) + 5 = 130$ parameters. `maxpool` has 0 parameters. $\texttt{affine}_1$ has $845 * 128 + 128 = 108288$ parameters.

2. **Neural network with weights and biases norm constraints** (31 points)

Consider the neural network with the following architecture

$$\begin{aligned} h_1 &= W_1 x^{(i)} + b_1 \\ a_1 &= \text{softplus}(h_1) \\ h_2 &= W_2 a_1 + b_2 \\ \hat{y} &= \sigma(h_2) \\ L &= \sum_{i=1}^{K} \max(0, 1 - y_i \hat{y}_i) \end{aligned}$$

where $x^{(i)} \in \mathbb{R}^{D_x \times 1}, W_1 \in \mathbb{R}^{D_{a_1} \times D_x}, W_2 \in \mathbb{R}^{K \times D_{a_1}}$. σ is the sigmoid activation function and y is a K-dimensional vector of 1's and 0's. y_i represents the ith element of y and $\hat{y}_i$ represents the ith element of $\hat{y}$. Softplus is a smooth approximation to the ReLU function and can be used to constrain the output to always be positive. It is an element-wise function defined as follows:

$$\text{softplus}(x) = \frac{1}{\beta} \log(1 + e^{\beta x})$$

In this question, you need to be very precise with the order of operations and transposes. If you miss a transpose or swap the order of a matrix vector multiply, you will not receive points for this question. You should also specify elementwise multiplication (i.e., the Hadamard product) with the symbol: $\odot$. You can use any gradients or backprop gradients we derived in class without proof.

(a) (3 points) Draw the computational graph for the neural network architecture described above. In the computational graph, you can write softplus as a single node (i.e., you do not have to draw the graph for $\frac{1}{\beta} \log(1 + e^{\beta x})$).

Solution:

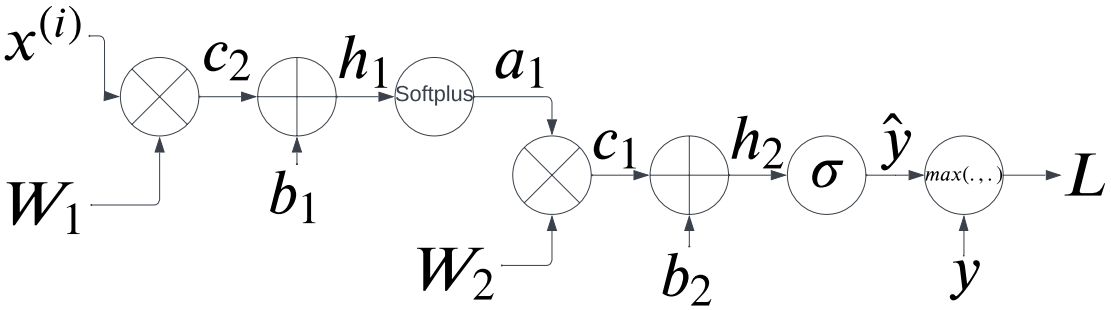


Figure 1: Computational graph

(b) (3 points) Compute $\nabla_{\hat{y}} L$. For all the following parts, you can refer to this computed gradient as $\delta_{\hat{y}}$.

Solution: From problem statement,

$$L = \sum_{i=1}^{K} \max(0, 1 - y_i \hat{y}_i).$$

Hence,

$$\nabla_{\hat{y}_i} L = \begin{cases} -y_i, & 1 - y_i \hat{y}_i \geq 0 \\ 0, & 1 - y_i \hat{y}_i < 0. \end{cases}$$

Since, $\hat{y}$ is the output of a sigmoid operator so $\hat{y}_i < 1$. Similarly, from problem statement $y_i \leq 1$. Hence, $1 - y_i \hat{y}_i \geq 0$. Therefore,

$$\nabla_{\hat{y}_i} L = -y_i.$$

Hence,

$$\delta_{\hat{y}} = -y$$

(c) (3 points) Compute $\nabla_{h_2} L$. For all the following parts, you can refer to this computed gradient as δ_{h_2}. Recall the gradient $\nabla_x \sigma(x) = \sigma(x)(1 - \sigma(x))$.

Solution: By chain rule,

$$\nabla_{h_2} L = \nabla_{h_2} \hat{y} \times \nabla_{\hat{y}} L$$

Since, $\sigma(.)$ is an elementwise operator so

$$\delta_{h_2} = -\hat{y} \odot (1 - \hat{y}) \odot y$$

(d) (3 point) Compute $\nabla_{W_2} L$. For all the following parts, you can refer to this computed gradient as δ_{W_2}.

Solution: Locally,

$$c_1 = W_2 a_1.$$

Using chain rule,

$$\nabla_{W_2} L = \nabla_{W_2} c_1 \times \nabla_{c_1} L.$$

Since $\nabla_{W_2} c_1$ is a 3-D tensor so using tensor-derivative trick,

$$\delta_{W_2} = \delta_{h_2} a_1^T$$

(e) (1 point) Compute $\nabla_{b_2} L$. For all the following parts, you can refer to this computed gradient as δ_{b_2}.

Solution: Since + gate passes the gradient, so

$$\delta_{b_2} = \delta_{h_2}$$

(f) (3 points) Compute $\nabla_{a_1} L$. For all the following parts, you can refer to this computed gradient as δ_{a_1}.

Solution: Using chain rule,

$$\nabla_{a_1} L = \nabla_{a_1} c_1 \times \nabla_{c_1} L.$$

Since,

$$\nabla_{a_1} c_1 = W_2^T,$$

so

$$\delta_{a_1} = W_2^T \delta_{h_2}$$

(g) (3 points) Compute $\nabla_{h_1} L$. For all the following parts, you can refer to this computed gradient as δ_{h_1}.

Solution: From the computational graph,

$$a_1 = \text{softplus}(h_1)$$

Since softplus like $ReLU$ is an elementwise operation so backpropagating through it leads to a hadamard product

$$\delta_{h_1} = \left(\frac{e^{\beta h_1}}{1 + e^{\beta h_1}} \right) \odot \left(\delta_{a_1} \right)$$

(h) (3 points) Compute $\nabla_{W_1} L$. For all the following parts, you can refer to this computed gradient as δ_{W_1}.

Solution: Locally,

$$c_2 = W_1 x^{(i)}$$

Using chain rule,

$$\nabla_{W_1} L = \nabla_{W_1} c_2 \times \nabla_{c_2} L.$$

Since $\nabla_{W_1} c_2$ is a 3-D tensor so using tensor-derivative trick,

$$\delta_{W_1} = \delta_{h_1} {x^{(i)}}^T$$

(i) (1 point) Compute $\nabla_{b_1} L$. For all the following parts, you can refer to this computed gradient as δ_{b_1}.

Solution: Since + gate passes the gradient, so

$$\delta_{b_1} = \delta_{h_1}$$

(j) (8 points) Based on some apriori information, you want to impose some norm constraints on W_1, W_2 and b_1. After exploring the various norm constraints, you have decided to modify the loss function in the following way:

$$\tilde{L} = L + LSE(b_1) + \|W_1\|_\infty + \|W_2\|_F^2,$$

where Log-Sum-Exponent (LSE) of a vector $x \in \mathbb{R}^n$ is defined as follows:

$$LSE(x) = \ln\left(\sum_{i=1}^{n} e^{x_i}\right).$$

Also, recall for $A \in \mathbb{R}^{m\times n}$

$$\|A\|_\infty = \max_{1\le i\le m} \sum_{j=1}^{n} |a_{ij}|$$
$$\|A\|_F^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2$$

Compute $\nabla_{W_1}\tilde{L}$, $\nabla_{W_2}\tilde{L}$ and $\nabla_{b_1}\tilde{L}$. **Note**: your answer should include terms $\nabla_{W_1}L$, $\nabla_{W_2}L$, and $\nabla_{b_1}L$, respectively, as these terms cannot be simplified.

Solution: Since gradient is a linear operator, so

$$\nabla_{W_1}\tilde{L} = \nabla_{W_1}L + \nabla_{W_1}\|W_1\|_\infty$$
$$\nabla_{W_2}\tilde{L} = \nabla_{W_2}L + \nabla_{W_2}\|W_2\|_F^2$$
$$\nabla_{b_1}\tilde{L} = \nabla_{b_1}L + \nabla_{b_1}LSE(b_1)$$

Now, let j denote the index of the row of W_1 with maximum row sum, then

$$\nabla_{W_1}\|W_1\|_\infty = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \text{sign}(w_j) \\ \mathbf{0} \end{bmatrix}$$

Similarly,

$$\nabla_{b_1}LSE(b_1) = \begin{bmatrix} \frac{e^{x_1}}{\sum_{i=1}^{D_{a_1}} e^{x_i}} \\ \vdots \\ \frac{e^{x_{D_{a_1}}}}{\sum_{i=1}^{D_{a_1}} e^{x_i}} \end{bmatrix}^T$$
$$= \text{softmax}(b_1)$$

Similarly,

$$\nabla_{W_2}\|W_2\|_F^2 = 2W_2$$

Hence,

$$\nabla_{W_1}\tilde{L} = \nabla_{W_1}L + \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \text{sign}(w_j) \\ \mathbf{0} \end{bmatrix}$$

$$\nabla_{W_2}\tilde{L} = \nabla_{W_2}L + 2W_2$$

$$\nabla_{b_1}\tilde{L} = \nabla_{b_1}L + \text{softmax}(b_1)$$

3. **Elastic Net Regularization** (20 points)
In this question, we will discuss some characteristics of Elastic Net. Elastic Net Regression is an extension of linear regression that incorporates both $L1$ (Lasso) and $L2$ (Ridge) regularization penalties. The loss function is defined as:

$$\mathcal{L}(\theta) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \lambda \left(\alpha \|\theta\|_1 + \frac{(1-\alpha)}{2} \|\theta\|_2^2 \right)$$

where n is the number of data examples, $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{X} \in \mathbb{R}^{n \times d}$, and $\theta \in \mathbb{R}^d$ where d is the number of features.

(a) (4 points) Describe the role of α. If $\alpha \to 1$, which regularization method does Elastic Net reduce to? If $\alpha \to 0$, which regularization method does Elastic Net reduce to? Answer this question with no more than 3 sentences.
Solution:

α balances the $L1$ (sparsity) and $L2$ (shrinkage) penalties.

When $\alpha \to 1$, Elastic Net reduces to pure $L1$ regularization.

When $\alpha \to 0$, Elastic Net reduces to pure $L2$ regularization.

(b) (8 points) Briefly explain the advantages of Elastic Net compared to standalone $L1$ regression and standalone $L2$ regression. Answer this question with no more than 4 sentences.
Solution:

vs. $L1$: It avoids Lasso's instability caused by arbitrarily selecting one from a group of highly correlated features by adding L2 regularization, reducing random feature selection bias.

vs. $L2$: It inherits L1 sparsity, enabling feature selection (unlike regular Ridge which only shrinks coefficients).

(c) (8 points) Derive the gradient of the Elastic Net loss function with respect to the parameters θ, i.e. $\nabla_\theta \mathcal{L}(\theta)$.
Solution:

Derivative of the Residual Squared Term:

$$\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 = \frac{1}{2n} (\mathbf{y} - \mathbf{X}\theta)^T (\mathbf{y} - \mathbf{X}\theta)$$

The gradient with respect to θ:

$$\nabla_\theta \left(\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 \right) = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\theta)$$

Derivative of the L1 Regularization Term (notations align with Discussion 2):

$$\lambda\alpha \|\theta\|_1 = \lambda\alpha \sum_i |\theta_i|$$

$$\nabla_\theta (\lambda\alpha \|\theta\|_1) = \lambda\alpha \cdot \text{sign}(\theta)$$

where:

$$\text{sign}(\theta_i) = \begin{cases} 1 & \text{if } \theta_i > 0, \\ -1 & \text{if } \theta_i < 0, \\ 0 & \text{if } \theta_i = 0. \end{cases}$$

Derivative of the L2 Regularization Term:

$$\lambda \frac{(1-\alpha)}{2} \|\theta\|_2^2 = \lambda \frac{(1-\alpha)}{2} \theta^T \theta$$

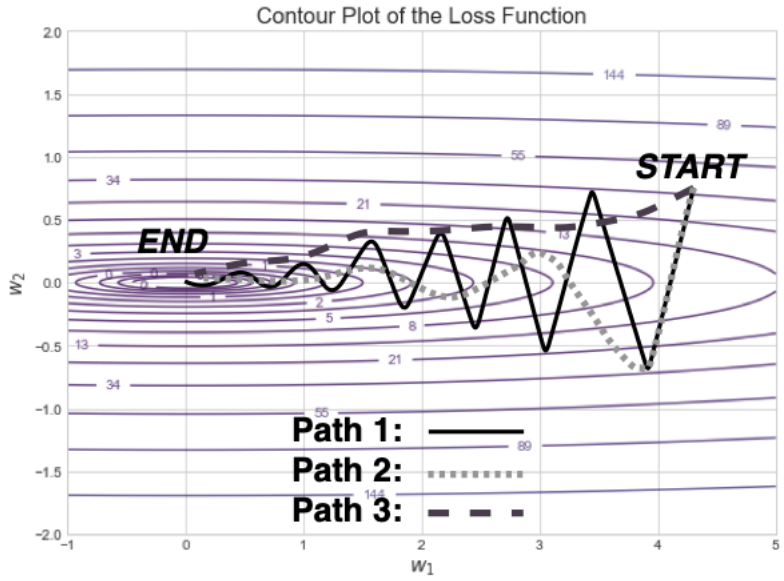
$$\nabla_\theta \left(\lambda \frac{(1-\alpha)}{2} \|\theta\|_2^2 \right) = \lambda(1-\alpha)\theta$$

Summing all three components:

$$\nabla_\theta \mathcal{L}(\theta) = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\theta) + \lambda \left(\alpha \cdot \text{sign}(\theta) + (1-\alpha)\theta \right)$$

4. **Optimization techniques** (25 points)

(a) (15 points) Consider a two-dimensional loss function $L(w_1, w_2)$ whose contour lines form a narrow valley primarily along the w_1 axis. Three distinct optimization trajectories are observed when minimizing L:



Match each trajectory shown in the plot (paths i, ii and iii) to one of the following optimizers:

SGD, SGD with momentum, RMSProp

Provide a brief justification (in no more than six sentences) for your matching.

Solution:

i. **SGD - path i**:SGD shows consistent progress along w_1 but shows significant oscillations along w_2, since there is no momentum or adaptive learning rate decay, the oscillation are predominantly present for a lot longer before eventually dampening as the model converges to the minima.

ii. **SGD with momentum - path ii**: Momentum accelerates progress along the flat w_1 direction but can cause overshooting in the steep w_2 direction leading to rapid advancement along w_1 with significant initial oscillations in w_2, eventually the dampening factor of the momentum helps in dampening the oscillations along w_2 before converging to the minima

iii. **RMSProp - path iii**: By adapting per-parameter learning rates based on gradient magnitudes without adding momentum, RMSprop yields a smooth trajectory that dynamically adjusts steps in both $w_1$and w_2 to follow the valley.

(b) (10 points) Given the RMSProp update equations:

$$\mathbf{v}_t = \gamma\, \mathbf{v}_{t-1} + (1-\gamma)\mathbf{g}_{t-1}^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\epsilon}{\sqrt{\mathbf{v}_t + \delta}} \odot \mathbf{g}_{t-1},$$

where we define $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_t)$ as the gradient of L at $\boldsymbol{\theta}_t$, derive an expression for $\boldsymbol{\theta}_2$ in terms of $\boldsymbol{\theta}_0$, $\mathbf{g}_0$, and $\mathbf{g}_1$. Your derivation should clearly show how the accumulated squared gradients affect the update.

Note: For a vector $\mathbf{v}$, the operations $\mathbf{v}^2$, $\sqrt{\mathbf{v}}$, and $1/\mathbf{v}$ are all element-wise operations.

Solution:

We start with the RMSProp update equations:

$$\mathbf{v}_t = \gamma\, \mathbf{v}_{t-1} + (1-\gamma)\mathbf{g}_{t-1}^2, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\epsilon}{\sqrt{\mathbf{v}_t + \delta}} \odot \mathbf{g}_{t-1},$$

with $\mathbf{v}_0 = 0$.

For $t = 1$:

$$\mathbf{v}_1 = (1-\gamma)\mathbf{g}_0^2, \quad \boldsymbol{\theta}_1 = \boldsymbol{\theta}_0 - \frac{\epsilon}{\sqrt{(1-\gamma)\mathbf{g}_0^2 + \delta}} \odot \mathbf{g}_0.$$

For $t = 2$:

$$\mathbf{v}_2 = \gamma\, \mathbf{v}_1 + (1-\gamma)\mathbf{g}_1^2 = \gamma(1-\gamma)\mathbf{g}_0^2 + (1-\gamma)\mathbf{g}_1^2,$$

and

$$\boldsymbol{\theta}_2 = \boldsymbol{\theta}_1 - \frac{\epsilon}{\sqrt{\mathbf{v}_2 + \delta}} \odot \mathbf{g}_1 = \boldsymbol{\theta}_1 - \frac{\epsilon}{\sqrt{(1-\gamma)[\gamma\mathbf{g}_0^2 + \mathbf{g}_1^2] + \delta}} \odot \mathbf{g}_1.$$

Substituting the expression for $\boldsymbol{\theta}_1$ into the update for $\boldsymbol{\theta}_2$, we obtain:

$$\boldsymbol{\theta}_2 = \boldsymbol{\theta}_0 - \frac{\epsilon}{\sqrt{(1-\gamma)\mathbf{g}_0^2 + \delta}} \odot \mathbf{g}_0 - \frac{\epsilon}{\sqrt{(1-\gamma)[\gamma\mathbf{g}_0^2 + \mathbf{g}_1^2] + \delta}} \odot \mathbf{g}_1.$$

Bonus: Generative Perspective of Softmax Classifier (7 points). **Please note that no partial credit is awarded for the bonus problem within each sub-question.**
For the classification task, a discriminative classifier directly models $P(y = c \mid \mathbf{x})$, including the Softmax classifier we covered in class:

$$P(y = c \mid \mathbf{x}) = \frac{\exp\left(\mathbf{w}_c^\top \mathbf{x} + b_c\right)}{\sum_{c'} \exp\left(\mathbf{w}_{c'}^\top \mathbf{x} + b_{c'}\right)}, \tag{1}$$

where $\mathbf{x} \in \mathcal{X}$ are the input and $y \in \mathcal{Y} = \{1, \cdots, C\}$ are the labels.

In contrast to the discriminative classifier, a generative classifier defines the class conditional distribution $P(\mathbf{x} \mid y)$ and class prior $P(y)$ to indirectly model $P(y \mid \mathbf{x})$ via specifying the joint distribution $P(\mathbf{x}, y) = P(y)P(\mathbf{x} \mid y)$.

One popular choice for a generative classifier is linear discriminant analysis (LDA). It assumes that the class conditional distribution $P(\mathbf{x} \mid y)$ follows the multivariate Gaussian distribution and the class prior $P(y)$ follows a discrete distribution:

$$P(\mathbf{x} \mid y = c) = \mathcal{N}\left(\mathbf{x} \mid \mu_c, \mathbf{\Sigma}\right), \quad P(y = c) = \frac{\beta_c}{\sum_{c'} \beta_{c'}}, \tag{2}$$

where μ_c and $\mathbf{\Sigma}$ are the mean and shared covariance of the multivariate Gaussian distribution, and β_c is the unnormalized prior for class c.

The multivariate Gaussian class conditional distribution for random variable $\mathbf{x} \in \mathbb{R}^D$ is defined as:

$$P(\mathbf{x} \mid y = c) = \frac{\exp\left(-\frac{1}{2}(\mathbf{x} - \mu_c)^{\mathrm{T}} \mathbf{\Sigma}^{-1} (\mathbf{x} - \mu_c)\right)}{\sqrt{(2\pi)^D |\mathbf{\Sigma}|}}, \tag{3}$$

where $|\mathbf{\Sigma}|$ denotes the determinant of the covariance matrix $\mathbf{\Sigma}$.

In this question, you will establish a connection between the generative LDA classifier and the discriminative Softmax classifier, providing a generative perspective on the latter.

1. (1 point) Express the posterior distribution $P(y = c \mid \mathbf{x})$ in terms of $P(y = c)$ and $P(\mathbf{x} \mid y = c)$, where $\mathbf{x} \in \mathcal{X}$ are the input and $y \in \mathcal{Y} = \{1, \cdots, C\}$ are the labels.

   Hint: Bayes' rule and marginalization

   Solution:

   $$P(y = c \mid \mathbf{x}) = \frac{P(y = c)\, P(\mathbf{x} \mid y = c)}{\sum_{c'} P\big(y = c'\big)\, P\big(\mathbf{x} \mid y = c'\big)}. \tag{4}$$

2. (5 points) Using the definitions of class prior $P(y = c)$ and class conditional distribution $P(\mathbf{x} \mid y = c)$ given in equation (2) and (3), show that your expression for $P(y = c \mid \mathbf{x})$ in part (a) simplifies to

   $$P(y = c \mid \mathbf{x}) = \frac{\exp\big(\mu_c^\top \mathbf{\Sigma}^{-1} \mathbf{x} - \frac{1}{2} \mu_c^\top \mathbf{\Sigma}^{-1} \mu_c + \log \beta_c\big)}{\sum_{c'=1}^{C} \exp\big(\mu_{c'}^\top \mathbf{\Sigma}^{-1} \mathbf{x} - \frac{1}{2} \mu_{c'}^\top \mathbf{\Sigma}^{-1} \mu_{c'} + \log \beta_{c'}\big)}. \tag{5}$$

   Note: log denotes natural logarithm

Hint 1: $x = \exp(\log(x))$

Hint 2: $\exp(a+b) = \exp(a)\exp(b)$

Solution: Directly plug in equation (2) and (3) to equation (4), we have

$$P(y=c \mid \mathbf{x}) = \frac{\beta_c \cdot \exp\left[-\frac{1}{2}(\mathbf{x}-\mu_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu_c)\right]}{\sum_{c'} \beta_{c'} \cdot \exp\left[-\frac{1}{2}(\mathbf{x}-\mu_{c'})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\mu_{c'})\right]} \tag{6}$$

Expand the multiplication terms, we have

$$P(y=c \mid \mathbf{x}) = \frac{\beta_c \cdot \exp\left[\mu_c^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_c^\top \boldsymbol{\Sigma}^{-1}\mu_c - \frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}\right]}{\sum_{c'} \beta_{c'} \cdot \exp\left[\mu_{c'}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_{c'}^\top \boldsymbol{\Sigma}^{-1}\mu_{c'} - \frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}\right]} \tag{7}$$

Factor β_c terms into the exponents and factor out $\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$, we have

$$P(y=c \mid \mathbf{x}) = \frac{\exp\left[\ln \beta_c\right] \cdot \exp\left[\mu_c^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_c^\top \boldsymbol{\Sigma}^{-1}\mu_c\right] \cdot \exp\left[-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}\right]}{\sum_{c'} \exp\left[\ln \beta_{c'}\right] \cdot \exp\left[\mu_{c'} \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_{c'}^\top \boldsymbol{\Sigma}^{-1}\mu_{c'}\right] \cdot \exp\left[-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}\right]} \tag{8}$$

Cancel out the $\exp\left[-\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}\right]$ terms and factor in the $\exp\left[\ln \beta_c\right]$ terms, we have the desired form:

$$P(y=c \mid \mathbf{x}) = \frac{\exp\left(\mu_c^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_c^\top \boldsymbol{\Sigma}^{-1}\mu_c + \log \beta_c\right)}{\sum_{c'} \exp\left(\mu_{c'}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}\mu_{c'}^\top \boldsymbol{\Sigma}^{-1}\mu_{c'} + \log \beta_{c'}\right)}. \tag{9}$$

3. (1 point) Show that the LDA posterior distribution in equation (5) can be converted to the form of the Softmax classifier in equation (1).

   In other words, express $\mathbf{w}_c$ and b_c in terms of μ_c, $\boldsymbol{\Sigma}$, and β_c. This indicates the equivalence between LDA and Softmax classifier and provides a generative perspective for the latter.

   Hint: You can still do this part without finishing part (b)

   Solution: Notice that the exponent in the LDA posterior has a linear term in $\mathbf{x}$ plus a constant term. Hence we can identify

$$\mathbf{w}_c \;=\; \boldsymbol{\Sigma}^{-1}\,\mu_c,$$

and

$$b_c \;=\; -\,\tfrac{1}{2}\,\mu_c^\top\,\boldsymbol{\Sigma}^{-1}\,\mu_c \;+\; \log \beta_c.$$